ajariwala1 / **Swift-basics**

Code Issues Pull requests Actions Projects Security and quality Insights



Swift-basics

Public

Notifications Fork 0 Star 0

About

Core Swift language concepts with examples in Xcode Playgrounds.

Readme

Activity

0 stars

1 watching

0 forks

Report repository

Releases

No releases published

Contributors 1



ajariwala1 Abhishek Jariwala

n 1 Branch 0 Tags Go to file Go to file Code



ajariwala1 Create README.md fb70adb · last year 1 Commit

README.md Create README.md last year

README

Swift-basics

Core Swift language concepts with examples in Xcode Playgrounds.

Swift Programming Fundamentals

This repository contains guided examples and explanations for learning Swift using Xcode Playgrounds. It is intended for students preparing to build iOS applications with UIKit.

Each section introduces a core concept with a concise explanation and a runnable Swift code snippet. Topics covered include:

- Variables and Constants
- Type Inference and Annotation
- String Interpolation
- Arrays and Dictionaries
- Optionals
- Control Flow (If/Else, Loops)
- Functions and Parameters
- Closures
- Tuples
- Typealiases

You can use this repository alongside Xcode to explore Swift interactively and build a strong foundation for app development.

Getting Started

To follow along:

1. Open **Xcode**.
2. Select **File > New > Playground**.
3. Choose the **Blank** template and name it `SwiftBasics`.
4. Save the file and begin experimenting with the examples provided below.

Variables and Constants

Swift uses two primary keywords to define values in memory:

- `var` creates a **variable** whose value can change.
- `let` creates a **constant** whose value cannot be modified once assigned.

Swift encourages the use of constants by default to make code safer and less error-prone.

```swift
var age = 25
age = 26

let year = 2025
year = 2000                        // This line will cause a compile-time error
```

Type Inference and Annotation

Swift uses type inference to automatically assign types based on the values you provide. This keeps code concise. However, you can also provide type annotations explicitly for clarity or when inference is not enough.

```swift
var city = "auburn"
var pi = 3.14

var country: String = "USA"
```

String Interpolation

String interpolation allows variables to be embedded directly inside string literals using `\()` syntax.

```swift
let country = "USA"
print("Hello, \(country)")
```

Arrays

An array stores multiple values of the same type in an ordered list. You can access elements by index, append new elements, and iterate through the array.

```swift
var numbers = [1, 2, 3]
numbers.append(4)
print(numbers[0])
```

Swift arrays are zero-indexed, meaning the first element is at position 0.

Dictionaries

Dictionaries store key-value pairs and allow you to look up a value by its associated key. They are useful for representing labeled data.

```swift
var numbers = [1: "One", 2: "Two", 3: "Three"]
numbers[4] = "Four"
print(numbers)
```

In this example, each number is mapped to its word form.

Optionals

Optionals represent a value that may or may not exist. They are declared using a `?` after the type.

```swift
var y: String?
print(y)

y = "abc"
print(y!)
```

Force unwrapping `(!)` should be avoided unless you are certain the value is not nil, as it can cause runtime crashes. Swift provides safer ways to handle optionals, such as optional binding (if let), which will be covered later.

Conditional Statements

Swift uses `if` and `else` for decision-making logic.

```swift
let number = 10

if number > 5 {
    print("The number is greater than 5.")
} else {
    print("The number is 5 or less.")
}
```

Conditional logic helps your program react to dynamic data.

Loops

Swift includes `for-in`, `while`, and `repeat-while` loops for performing repeated tasks.

- Use `for-in` to loop through each element in a collection, such as an array or a range. It is the most common and readable way to iterate over items.
- A `while` loop checks its condition before each iteration. If the condition is false at the start, the

loop body will not execute at all.

- A `repeat-while` loop runs the body once before checking the condition. It ensures the loop body executes at least once, regardless of the initial condition.

```
let numbers = [1, 2, 3]
for number in numbers {
    print(number)
}

var counter = 1
while counter <= 5 {
    print(counter)
    counter += 1
}

var count = 1
repeat {
    print(count)
    count += 1
} while count <= 3
```

Functions

Functions are reusable blocks of code. They can accept parameters, return values, and include external and internal parameter names.

```
func addNumbers(a: Int, b: Int) -> Int {
    return a + b
}

let result = addNumbers(a: 5, b: 3)
print(result)
```

Swift also supports default parameter values and omitting external parameter names using `_` .

```
func greet(name: String, message: String = "Welcome") {
    print("\(message), \(name)!")
}

greet(name: "Abhi")
greet(name: "Abhi", message: "Hi")
```

Closures

Closures are anonymous functions that can be stored in variables, passed as parameters, or returned from other functions.

```
let square = { (number: Int) -> Int in
    return number * number
}

print(square(4))
```

Swift supports several ways to simplify closure syntax using type inference, single-expression returns, and trailing closure syntax.

```
let square = { (number : int) in number * number }
```

Higher-Order Functions

Functions can accept other functions or closures as parameters. This is especially useful for custom logic, sorting, filtering, and callback mechanisms.

```
func operateOnNumbers(_ a: Int, _ b: Int, operation: (Int, Int) -> Int) -> Int {
    return operation(a, b)
}

let result = operateOnNumbers(5, 7) { x, y in x + y }
print(result)
```

Tuples

Tuples let you group multiple values together into a single compound value.

```
let person = (name: "Bob", age: 25)
print(person.name)
print(person.age)
```

Tuples are especially useful for returning multiple values from a function without creating a separate type.

```swift
func findMinMax(numbers: [Int]) -> (min: Int, max: Int) {
    let minValue = numbers.min() ?? 0
    let maxValue = numbers.max() ?? 0
    return (minValue, maxValue)
}
```

Typealiases

A `typealias` allows you to assign a custom name to an existing type. This improves readability, especially with complex types like closures or tuples.

```swift
typealias MathOperation = (Int, Int) -> Int

let add: MathOperation = { a, b in a + b }
print(add(5, 7))
```

What's Next

This module serves as your foundation in Swift. We've explored variables, constants, types, collections, conditionals, loops, functions, closures, tuples, and typealiases. These are the tools you will use throughout iOS development.

In the next module, you will use this knowledge to build your first UIKit-based application: a DiceRoller with real UI components, assets, and layout constraints.